

Trabajo Práctico Final

Sistema de Gestión de Pedidos, Inventario y Clientes (arquitectura de microservicios)

Fechas Relevantes y Condiciones

- **Fecha de entrega:**
 - Martes 24 de Noviembre.
- **Grupos:**
 - 3-4 integrantes
- **Coloquios:**
 - Fecha tentativa: Lunes 30 de noviembre
 - La fecha puede cambiar debido a que dependemos de la disponibilidad de aulas.
 - Se deberá realizar una demo del sistema funcionando. Cada integrante del grupo tendrá que llevar adelante una parte de la demo. También habrá preguntas de defensa.
- **Condiciones de aprobación:**
 - El trabajo práctico no se re-entrega: el mismo será desarrollado durante el cuatrimestre, por ende, al momento de entregar, todas las consignas deben funcionar. De no ser así, el trabajo no será aprobado y no se obtendrá la regularidad/promoción de la materia.
- **Uso de inteligencia artificial:**
 - Debido a que el uso de herramientas de AI para generar el código evitan que los conocimientos sean absorbidos por el alumno, queda prohibido el uso de las mismas dentro de la materia.
 - De detectarse el uso de las mismas, el trabajo será reprobado.

Alcance funcional (mínimo)

El trabajo abarca los contenidos completos de la materia. Cada unidad tendrá sus correspondientes consignas a llevar adelante:

1. **Migraciones de la base de datos:** los esquemas de bases de datos y otros cambios deseados deben realizarse utilizando *Flyway*.
2. **Diseño de API REST:** cada servicio debe exponer su correspondiente API para llevar adelante las tareas requeridas por la aplicación, siguiendo convenciones y buenas prácticas.
3. **Procesamiento por lotes:** se deberá implementar un job utilizando *SpringBatch*.
4. **Test Driven Development:** utilizar la estrategia en el desarrollo del trabajo, mediante el uso de herramientas vistas en clase.

5. **Release management:** se usarán feature flags para poder verificar distintos caminos de ejecución.
6. **Observabilidad:** se pide poder filtrar logs y seguir un trace.

Enunciado

TiendaOnline necesita modernizar su backend separando responsabilidades en tres microservicios: servicio-clientes (datos de clientes), servicio-inventario (gestión de stock) y servicio-pedidos (gestión de pedidos). El objetivo del TP es diseñar e implementar este backend aplicando las prácticas y herramientas vistas en la materia.

Requerimientos

servicio-clientes

Este servicio debe exponer los siguientes endpoints:

- Registrar un cliente.
- Consultar datos de un cliente por id.
 - Dentro de este endpoint los campos nombre y email (PII) deben poder enmascarse con asteriscos (ej. "J*** P****", "*****@****.com") según el estado de la feature flag *mask-pii*, evaluada en runtime; con el flag inactivo se devuelven sin enmascarar.
- Retornar los pedidos de un cliente determinado, que devuelve el listado de pedidos de un cliente consultando de forma síncrona a *servicio-pedidos*.

servicio-inventario

Este servicio debe exponer los siguientes endpoints:

- Consultar la disponibilidad de stock de un producto.
- Reservar unidades de stock para un pedido, rechazando la operación si no hay stock suficiente.
- Liberar una reserva existente.

servicio-pedidos

Este servicio debe exponer los siguientes endpoints:

- Crear un pedido: el endpoint debe:
 - verificar que exista el cliente (utilizando el *servicio-clientes*)
 - reservar el stock de cada ítem contra *servicio-inventario*
 - calcular el monto total.
 - Si el cliente no existe o no hay stock suficiente, el pedido no se crea y se devuelve el error correspondiente.
- Consultar un pedido mediante su identificador.
- Listar los pedidos de un cliente (esta operación la consume *servicio-clientes*).



- Cancelar un pedido en estado CREADO o CONFIRMADO, liberando la reserva de stock asociada en servicio-inventario.

Proceso por lotes

- Un batch job nocturno (Spring Batch) recorre los pedidos en estado CREADO con más de 24hs sin confirmar, los pasa a EXPIRADO y libera la reserva de stock correspondiente en servicio-inventario, generando un resumen de la corrida.

Modelo de datos (detalle de implementación)

Campos mínimos esperados por entidad; los alumnos pueden agregar los que consideren necesarios para su diseño.

- servicio-clientes — Cliente: id (entero autoincremental), nombre (PII), email (PII), fechaAlta.
- servicio-inventario — Producto: id (entero autoincremental), nombre, cantidadDisponible.
- Reserva: id (entero autoincremental), producto, pedido, cantidad, estado (ACTIVA, LIBERADA).
- servicio-pedidos — Pedido: id (entero autoincremental), clienteId, estado (CREADO, CONFIRMADO, CANCELADO, EXPIRADO), montoTotal, fechaCreacion, fechaActualizacion.
- ItemPedido: id (entero autoincremental), pedido, producto, cantidad, precioUnitario.

Requerimientos técnicos obligatorios

- Java + Spring Boot, arquitectura de al menos tres microservicios independientes, cada uno con su propia base de datos.
- API REST documentada (OpenAPI/Swagger), con validaciones y manejo de errores consistente.
- Flyway para versionar el esquema de cada base de datos. **No se admite ddl-auto=update.**
- Spring Batch para el job de expiración de pedidos, con al menos un Step de tipo chunk (reader/processor/writer).
- Feature flags (Statsig) para el flag *mask-pii*, evaluado en runtime.
- Tests Unitarios:
 - uso de Mockito para tests unitarios.
- Tests de Integración:
 - uso de MockServer para simular las llamadas entre servicios y *@SpringBootTest*.
- Observabilidad:
 - Mínimo: observar logs, buscar logs, traces.
 - Bonus: cualquier otra característica que pueda agregarse y mostrarse en la demo.



Entregables

1. Repositorio (fork del esqueleto) con los servicios implementados.
2. Documento que contenga:
 - a. Carátula indicando los integrantes del grupo.
 - b. Decisiones de diseño que se hayan tomado y deseen aclarar.
 - c. URL del repositorio entregado, y cualquier aclaración que quieran agregar.